

Ea-Nasir Travel

A polyglot architecture booking engine

Stefano Pio Lazzara - 566085

June 3, 2026

Contents

1	The old project and its weaknesses	2
1.1	The limits of legacy SQL	2
1.2	Inspiration for the new name	3
2	The polyglot architecture	4
2.1	Docker environment setup	4
2.2	MariaDB and session state	4
2.3	MongoDB and polymorphic tickets	4
3	Scalability and Vision	6
3.1	Multilanguage Interface and API Limitations	6
3.2	Expandable Components	6

Chapter 1

The old project and its weaknesses

This project is born as an evolution, expanding both the software stack and the underlying database schema.

The original project functioned primarily as a static request interface. It allowed users to state their desire to travel from one place to another on a specific date, and utilized a many-to-many (M:M) relational structure to cross-reference their passports against destination visa requirements. While this successfully implemented the M:M relationship required by previous academic guidelines, it operated merely as a digital form for a human travel agent. It did not account for real-world constraints such as flight times, pricing, or carrier selection.

Furthermore, the initial project was constrained by mandatory entity thresholds and an obligation to strictly utilize a Relational Database Management System (RDBMS). By removing these constraints, this new iteration adopts a modern and Object-Oriented Programming approach using PHP and JavaScript. Most importantly, it transitions to a *polyglot persistence* model, integrating both structured SQL data and unstructured NoSQL data (JSON via MongoDB) to handle complex travel itineraries.

Functionally, the scope has been changed. Rather than attempting to manage an unrealistic array of transportation methods, the application now acts as a dedicated, real-time flight booking engine powered by the Duffel API.

1.1 The limits of legacy SQL

While the application architecture has grown in complexity, the core database philosophy remains focused on selecting the appropriate tool for each specific data domain. To ensure secure, standardized, and reliable user registration, the *Users* entity is managed via a structured SQL database (MariaDB). A relational model is ideal for core user credentials because the data schema is highly predictable.

However, this hybrid architecture also leaves room for flexible future expansions, such as integrating passport data. Because a user might possess zero, one, or multiple passports, this data is an ideal candidate for JSON implementation rather than rigid relational tables. Utilizing a JSON array to store passport information would easily manage these variable quantities without requiring complex join tables. This data could then eventually power an advanced filtering engine that recommends accessible destinations based on a user's specific citizenships and visa requirements.

Conversely, flight tickets represent highly variable, unpredictable data. In a legacy SQL system, handling a round-trip ticket versus a one-way ticket within the exact same table creates significant architectural inefficiencies. A traditional relational table would require dedicating multiple strict columns to the return flight (e.g., `return_departure_date`, `return_arrival_airport`). If a customer books a simple one-way ticket, all of these columns would remain entirely empty

(NULL).

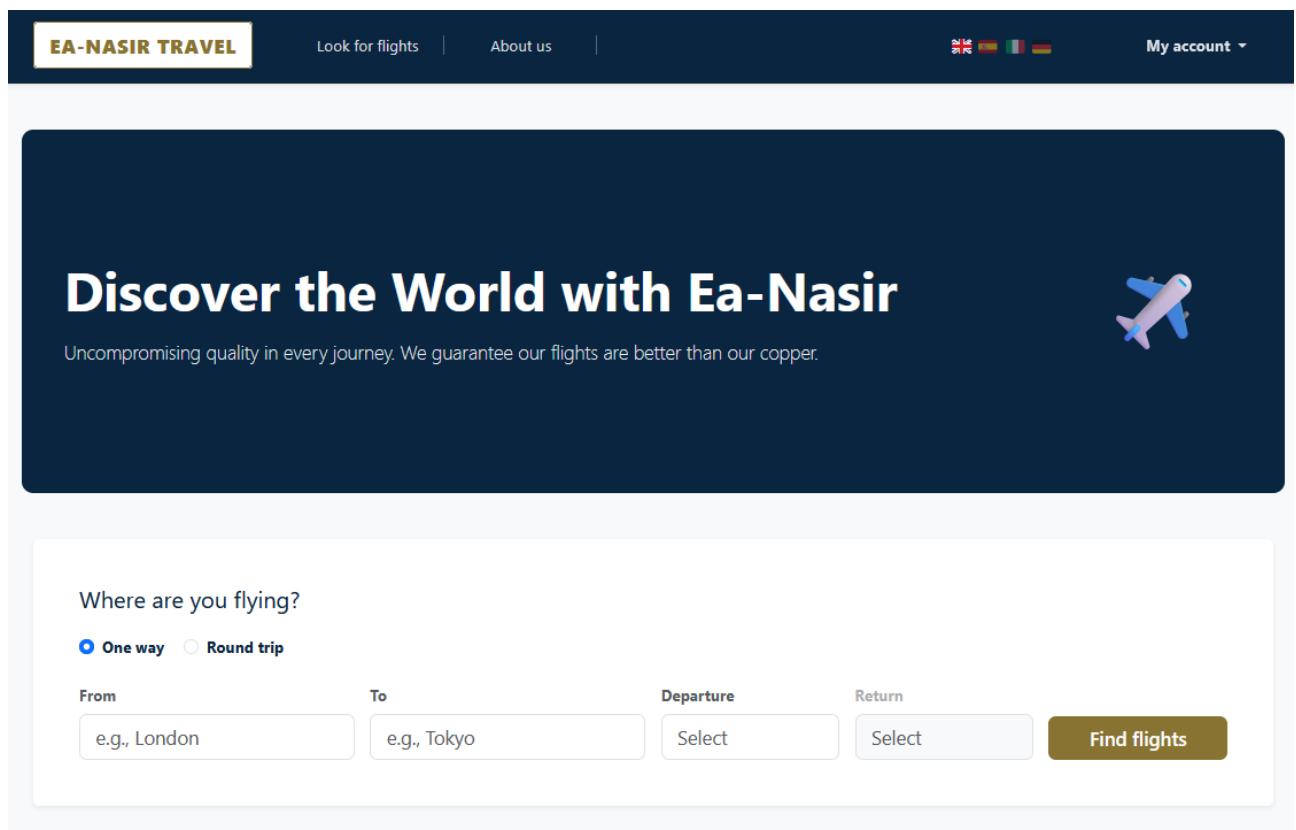
At scale, this leads to "sparse data" and wasted database memory. By migrating the ticket system to a NoSQL document-based structure, the application circumvents this limitation. A JSON document dynamically shapes itself to include a return flight object *only* when one is actually booked, completely eliminating the inefficiencies of legacy SQL schema rigidity.

1.2 Inspiration for the new name

The name "Ea-Nasir Travel" stems from a combination of geohistorical context, personal heritage, and a touch of irony.

On a personal level, the nomenclature pays homage to my father's former travel agency, *Babilonia Travel*. This established a thematic foundation rooted in ancient Mesopotamia. The specific addition of "Ea-Nasir" references the infamous ancient Babylonian merchant, recognized by historians as the recipient of the world's oldest surviving written customer complaint. According to the artifact—a clay cuneiform tablet authored by a customer named Nanni—Ea-Nasir was accused of delivering substandard, low-quality copper and treating the customer's servant with contempt.

This theme is subtly shown throughout the application's user interface, which utilizes a primary color palette of deep blue and white, highlighted by distinct brown accents, especially shades closer to copper. Furthermore, because this web application operates as an academic simulation and does not process actual commercial airline tickets, the association with history's most notoriously unreliable merchant was chosen to highlight the project's theoretical nature while giving the platform a unique, memorable identity.



The screenshot displays the Ea-Nasir Travel website. The header features the logo "EA-NASIR TRAVEL" in a dark blue box, with navigation links "Look for flights" and "About us". On the right, there are flags for the United Kingdom, Germany, and Italy, and a "My account" link. The main content area has a dark blue background with the headline "Discover the World with Ea-Nasir" and a subtext "Uncompromising quality in every journey. We guarantee our flights are better than our copper." A small airplane icon is on the right. Below this is a white search form titled "Where are you flying?". It includes radio buttons for "One way" (selected) and "Round trip". The form has four input fields: "From" (with "e.g., London"), "To" (with "e.g., Tokyo"), "Departure" (with "Select"), and "Return" (with "Select"). A "Find flights" button is on the right.

Chapter 2

The polyglot architecture

At the core of this web application is the concept of *polyglot persistence*—the strategic practice of utilizing multiple distinct database technologies within a single system. Rather than forcing all data to conform to a single structural paradigm, this architecture routes data to the most appropriate storage medium based on its inherent shape, predictability, and usage requirements.

2.1 Docker environment setup

To guarantee reproducibility and eliminate the classic “it works on my machine” deployment dilemma, the entire application architecture is containerized using Docker. This setup isolates the web server, the relational database, and the NoSQL document store into distinct, interconnected microservices that communicate securely across a private internal network.

Specifically, the core application relies on a custom-built PHP container. Through a tailored `Dockerfile`, necessary system dependencies and extensions—such as Composer for package management and the native MongoDB driver for NoSQL communication—are injected directly into the environment at build time. This ensures that the application is fully portable, consistent, and instantly deployable in any modern hosting environment without requiring manual server configuration.

2.2 MariaDB and session state

MariaDB serves as the foundational relational database, exclusively tasked with handling user authentication, profile management, and session state. Because user credentials and access roles adhere to a strict, highly predictable schema, a Relational Database Management System (RDBMS) provides the necessary data integrity and security.

The application interacts with this relational data entirely through an object-oriented paradigm. Rather than utilizing legacy procedural scripts scattered throughout the frontend pages, dedicated classes encapsulate the database logic. A secure `User` object natively manages the connection to MariaDB, safely authenticates credentials, and initializes the secure session state. This encapsulation keeps the session protected and ensures that the UI cleanly accesses only the profile data permitted for the actively logged-in user.

2.3 MongoDB and polymorphic tickets

While structured user data is anchored in MariaDB, the core booking engine utilizes MongoDB to store the generated flight tickets. Because travel itineraries are highly variable in their struc-

ture, MongoDB's flexible, document-based NoSQL architecture is vastly superior to relational tables for this specific domain.

When a user confirms a booking, the system generates a polymorphic JSON document. If the flight is a one-way trip, the document contains only the outbound flight details. If it is a round-trip, the document dynamically expands its internal shape to include a nested return flight object. By storing this unstructured data in MongoDB, the application completely bypasses the sparse data and empty-column limitations of traditional SQL discussed in the previous chapter, resulting in a lean, efficient, and highly adaptable booking ledger.

Results for CTA to CDG (05/06/2026 - 15/06/2026)

Duffel Airways Outbound CTA 03:05 → CDG 05:50 Inbound CDG 10:50 → CTA 13:35	143.44 EUR Total combined price Select Ticket
Iberia Outbound CTA 03:05 → CDG 05:50 Inbound CDG 10:50 → CTA 13:35	144.01 EUR Total combined price Select Ticket
British Airways Outbound CTA 03:05 → CDG 05:50 Inbound CDG 10:50 → CTA 13:35	145.67 EUR Total combined price Select Ticket
Lufthansa Outbound CTA 06:00 → CDG 11:15 Inbound CDG 06:25 → CTA 11:00	165.24 EUR Total combined price Select Ticket

Ticket: ef01fec2 Confirmed Delete

PASSENGER PURCHASED ON
Salvatore Micheal Sorrentino 03 Jun 2026, 08:03

Round trip

Outbound flight Flight: EA9032
REG 2026-06-04 → **CDG**

Chapter 3

Scalability and Vision

To maintain focus on the core database architecture and the polyglot persistence model, certain frontend features and secondary functionalities were intentionally scoped out of this initial release. However, the foundational architecture was specifically designed to support seamless future integrations. The current infrastructure leaves ample room for expansion, including extending the SQL *Users* entity with JSON attributes, integrating new language tables, and enhancing the personal profile UI.

3.1 Multilanguage Interface and API Limitations

Drawing from my academic background in linguistics (graduating with a degree in Languages and holding three certifications), implementing international accessibility was a priority for this project. Ensuring a dynamic multilanguage interface was much easier thanks to the Object-Oriented Programming (OOP) approach, specifically through a custom `Translator` class that acts as a bridge between the backend and the frontend.

Initially, the localization scope was intended to include French and Russian. However, integrating Russian presented a complex technical conflict with the Duffel API. The API natively returns geographical data (such as city names and airports) in the Latin alphabet. For the currently supported languages, injecting these English API strings directly into the UI remains highly readable for the user. Conversely, Russian utilizes the Cyrillic alphabet; fully supporting it would have required building a massive intermediary translation matrix to map at least hundreds of English API responses to Cyrillic strings in real-time. To maintain search performance and application speed, this feature was tabled for future development.

3.2 Expandable Components

Several key components have been identified for future iteration. Chief among these is the enhancement of the user profile system to include the aforementioned JSON-based passport arrays. This data would enable an advanced algorithmic filter, allowing users to search for flights by destination country while automatically cross-referencing their specific visa requirements.

Furthermore, on the database level, the NoSQL architecture could be enhanced by implementing MongoDB Time-To-Live (TTL) indexes. This would automate data lifecycle management, allowing the server to automatically purge expired temporary reservation holds or past flights without requiring custom background scripts, ensuring the database remains highly performant at scale.